

LAB01

Compte rendu Déploiement et exploration d'une infrastructure OpenStack

Installation de DevStack sur AWS EC2 et gestion de machines virtuelles avec Horizon et la CLI

CONTENU DU DOCUMENT

- 01 Contexte et objectifs
- 02 Infrastructure AWS : l'hôte EC2
- 03 Installation et vérification de DevStack
- 04 Création d'une VM avec Horizon
- 05 Création d'une VM avec la CLI OpenStack
- 06 Analyse et conclusion

RÉALISÉ PAR

Ons

COURS

Infrastructure Cloud

DATE

Octobre 2026

00 Contexte et objectifs

Ce compte rendu documente le **LAB01 — Déploiement et exploration d'une infrastructure OpenStack**. Le principe : une instance AWS EC2 sert d'hôte unique pour une plateforme OpenStack installée avec **DevStack**. Sur cette plateforme, on crée ensuite des machines virtuelles de deux façons, avec l'interface **Horizon** puis avec la CLI `openstack`.

Le compte rendu suit la démarche des deux guides du TP (guide d'installation EC2 et énoncé OpenStack) :

Partie	Contenu	Guide
A	Provisionnement EC2, IP élastique, virtualisation imbriquée, connexion SSH	Guide EC2 §1-4
B	Installation de DevStack, vérification des services, accès à Horizon	Guide EC2 §5-7
C	Création d'une VM avec Horizon (réseau, routeur, sécurité, clés, IP flottante)	Énoncé §4-5
D	Création d'une VM avec la CLI OpenStack	Énoncé §6
E	Analyse, bonnes pratiques et conclusion	—

Environnement

Hôte : instance EC2 `DevStack` de type `r8i.large` (2 vCPU, 16 Go de RAM), région `us-east-1`, adresse privée VPC `172.31.43.124`. OpenStack : DevStack, avec Nova, Neutron, Glance, Cinder, Keystone et Horizon.

A Infrastructure AWS : l'hôte EC2

A.1 Provisionnement de l'instance

L'instance est créée selon les paramètres de l'énoncé. La capture de la console EC2 confirme que l'instance `DevStack` est en cours d'exécution avec le type `r8i.large`, dans la zone de disponibilité `us-east-1b`.

Paramètre	Valeur
Nom	<code>DevStack</code>
AMI	Ubuntu 26.04 LTS
Type d'instance	<code>r8i.large</code> (2 vCPU, 16 Go RAM)
Zone de disponibilité	<code>us-east-1b</code>
Paire de clés	<code>StackKey</code> (RSA, créée le 29/09/2026)
Groupe de sécurité	<code>DevStack SG</code> (SSH 22 et HTTP 80)
Stockage	1 volume EBS gp3 de 50 Gio

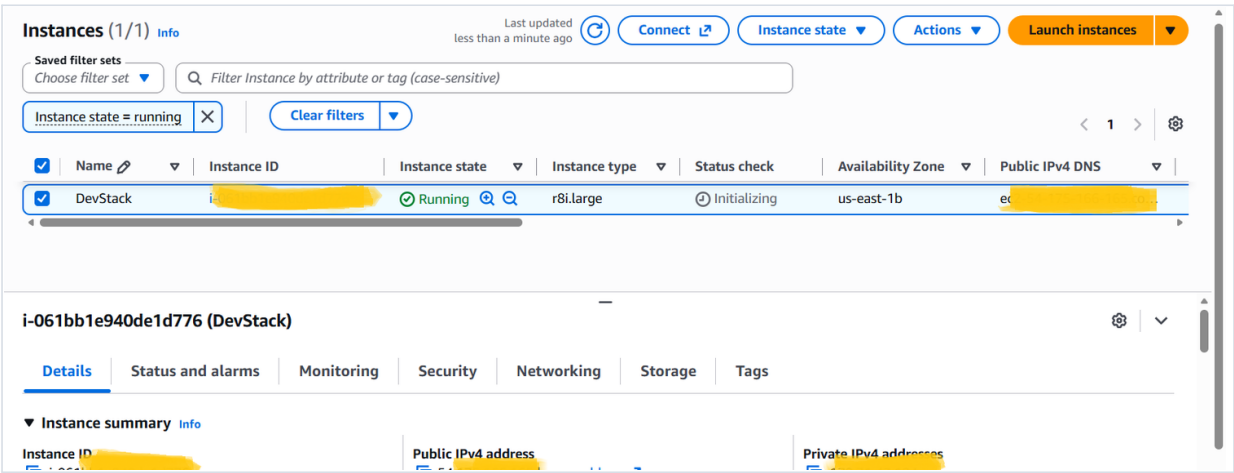


Figure 1 — Instance DevStack (r8i.large) en cours d'exécution dans la console EC2

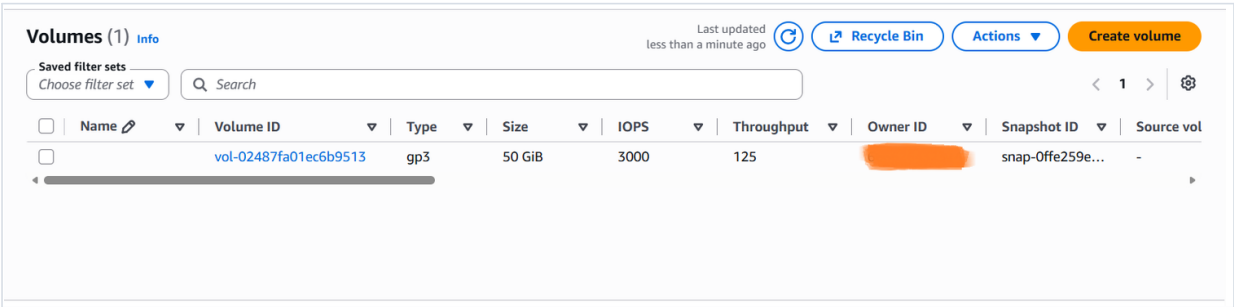


Figure 2 — Volume EBS de 50 Gio (gp3) attaché à l'instance

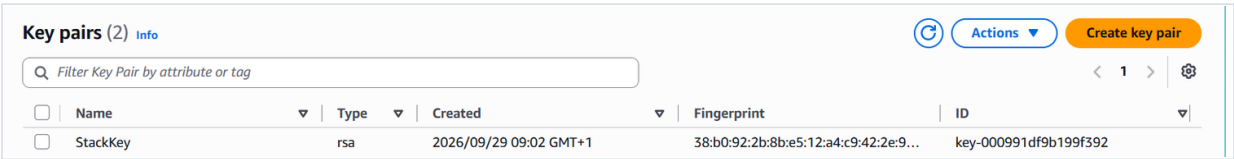


Figure 3 — Paire de clés RSA utilisée pour la connexion SSH

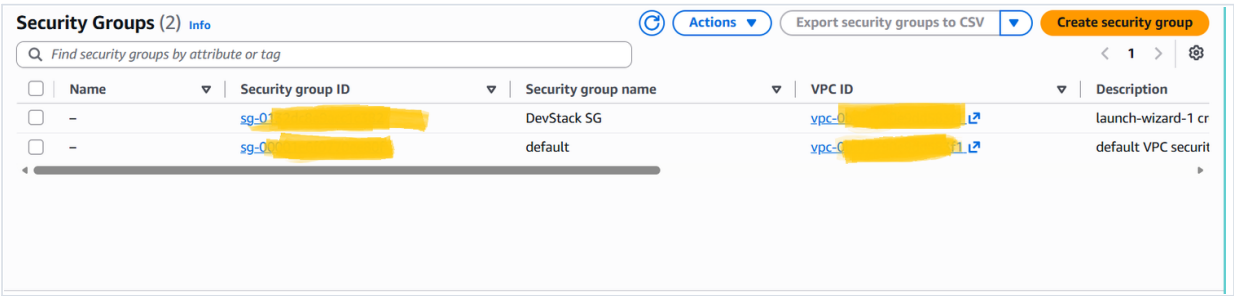


Figure 4 — Groupe de sécurité DevStack SG créé pour l'instance

Le groupe **DevStack SG** autorise en entrée **SSH (22)** pour l'administration et **HTTP (80)** pour accéder au dashboard Horizon.

Type	Protocole	Port	Source
SSH	TCP	22	0.0.0.0/0
HTTP	TCP	80	0.0.0.0/0

A.2 Adresse IP élastique

Une IP publique ordinaire change à chaque arrêt de l'instance. Or l'instance doit être arrêtée pour activer la virtualisation imbriquée : une **IP élastique** est donc allouée (**EC2 > Elastic IPs > Allocate**) puis associée à l'instance (**Actions > Associate Elastic IP address**). L'adresse publique reste ainsi stable pour la connexion SSH et pour l'URL de Horizon.

Contrôle depuis l'instance : l'adresse publique vue de l'extérieur doit être l'IP élastique.

COMMANDE

```
curl -s http://checkip.amazonaws.com
```

RÉSULTAT ATTENDU

```
<IP_élastique>
```

A.3 Activation de la virtualisation imbriquée

Nova délègue la création des VM à **KVM**, qui exige les extensions matérielles du CPU (Intel VT-x). L'hôte étant lui-même une VM AWS, ces extensions doivent lui être exposées. Procédure suivie :

1. Arrêter l'instance (**Stop**, pas **Terminate**).
2. **Actions > Instance settings > Change CPU options > Nested virtualization > Enable**.
3. Redémarrer l'instance et se reconnecter.
4. Contrôler avec `sudo kvm-ok`.

VÉRIFICATION

```
lscpu | grep -i virtualization
sudo apt update && sudo apt install -y cpu-checker
sudo kvm-ok
```

RÉSULTAT ATTENDU DE SUDO KVM-OK

```
INFO: /dev/kvm exists
KVM acceleration can be used
```

A.4 Connexion SSH

La connexion se fait avec la clé privée de l'instance :

POSTE LOCAL

```
chmod 400 "<cle>.pem"
ssh -i "<cle>.pem" ubuntu@<IP_élastique>
```

La commande `ip a` exécutée sur l'hôte confirme la session : l'interface `enp39s0` porte l'adresse privée VPC `172.31.43.124/20`.

```
ubuntu@ip-172-31-43-124:~/devstack$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp39s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc mq state UP group default qlen 1000
    link/ether 0e:ff:ef:c0:02:4f brd ff:ff:ff:ff:ff:ff
    altname enx0effefc0024f
    inet 172.31.43.124/20 metric 100 brd 172.31.47.255 scope global dynamic enp39s0
        valid_lft 2700sec preferred_lft 2700sec
    inet6 fe80::cff:efff:fec0:24f/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
```

Figure 5 — Session SSH sur l'hôte : interface enp39s0 (172.31.43.124/20)

B Installation et vérification de DevStack

B.1 Installation

Après mise à jour du système et installation des outils de base, DevStack est cloné et configuré avec le fichier `local.conf` fourni par l'énoncé, puis lancé avec `stack.sh`.

SÉQUENCE D'INSTALLATION

```
sudo apt update && sudo apt upgrade -y && sudo reboot
sudo apt install -y git curl net-tools
git clone https://opendev.org/openstack/devstack
cd devstack && nano local.conf
./stack.sh
```

LOCAL.CONF

```
[[local|localrc]]
ADMIN_PASSWORD=secret
DATABASE_PASSWORD=$ADMIN_PASSWORD
RABBIT_PASSWORD=$ADMIN_PASSWORD
SERVICE_PASSWORD=$ADMIN_PASSWORD
```

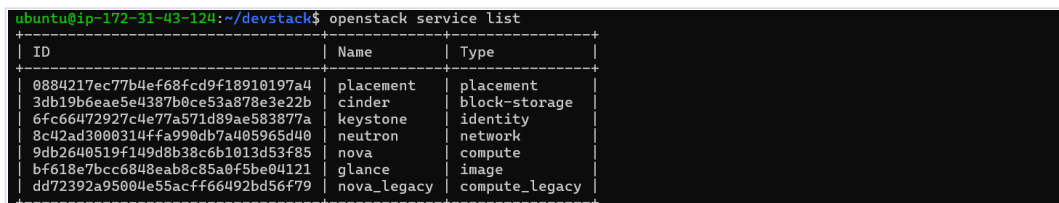
L'installation se termine par un message de fin :

RÉSULTAT ATTENDU DE `./STACK.SH`

```
stack.sh completed in <N> seconds.
```

B.2 Vérification des services

Les identifiants sont chargés avec `source openrc admin admin`, puis `openstack service list` interroge le catalogue de Keystone.



```
ubuntu@ip-172-31-43-124:~/devstack$ openstack service list
+-----+-----+-----+
| ID                                           | Name      | Type      |
+-----+-----+-----+
| 0884217ec77b4ef68fcd9f18910197a4          | placement | placement |
| 3db19b6eae5e4387b0ce53a878e3e22b          | cinder    | block-storage |
| 6fc66472927c4e77a571d89ae583877a          | keystone  | identity   |
| 8c42ad3000314ffa990db7a405965d40          | neutron   | network    |
| 9db2640519f149d8b38c6b1013d53f85          | nova      | compute    |
| bf618e7bcc6848eab8c85a0f5be04121          | glance    | image      |
| dd72392a95004e55acff66492bd56f79          | nova_legacy | compute_legacy |
+-----+-----+-----+
```

Figure 6 — Résultat de `openstack service list`

Service	Type	Rôle
<code>keystone</code>	identity	Authentification, projets, catalogue de services
<code>nova</code>	compute	Cycle de vie des machines virtuelles
<code>neutron</code>	network	Réseaux, sous-réseaux, routeurs, IP flottantes
<code>glance</code>	image	Catalogue d'images (cirros)
<code>cinder</code>	block-storage	Volumes de stockage en mode bloc
<code>placement</code>	placement	Inventaire et allocation des ressources
<code>nova_legacy</code>	compute_legacy	Point d'accès de compatibilité de Nova

Les sept services attendus sont présents : l'installation est complète.

B.3 Accès au dashboard Horizon

Horizon est accessible sur le port 80 de l'IP élastique de l'instance. Connexion avec l'utilisateur `admin` et le mot de passe défini dans `local.conf`.

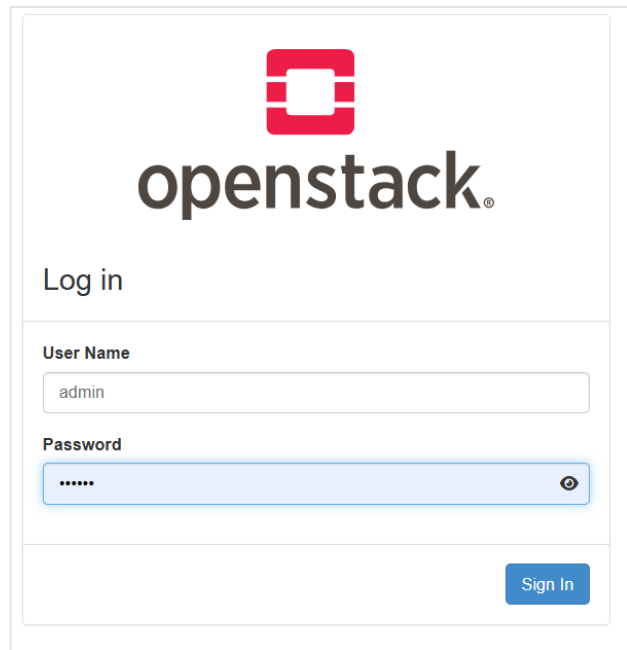


Figure 7 — Page de connexion du dashboard Horizon

C Création d'une VM avec Horizon

La plateforme est utilisée à travers le projet `demo`. Les ressources suivantes sont créées dans l'ordre, car chacune dépend de la précédente :

Ressource	Chemin dans Horizon	Valeur
Réseau et sous-réseau	Network > Networks > Create Network	<code>horizon-network</code> , <code>horizon-subnet</code> , <code>192.168.10.0/24</code>
Routeur	Network > Routers > Create Router	<code>horizon-router</code> , passerelle externe <code>public</code> , interface sur <code>horizon-subnet</code>
Groupe de sécurité	Network > Security Groups	<code>horizon-sg</code> , règle SSH (22)
Paire de clés	Compute > Key Pairs	<code>horizon-keypair</code>
Instance	Compute > Instances > Launch Instance	<code>horizon-vm</code> , image <code>cirros</code> , flavor <code>m1.tiny</code>
IP flottante	Network > Floating IPs	Pool <code>public</code> , associée à <code>horizon-vm</code>

Vérification de l'instance en CLI (statut `ACTIVE`, IP fixe `192.168.10.210` et IP flottante `172.24.4.92`) :

COMMANDE

```
source ~/devstack/openrc admin admin
openstack server list --all-projects
```

RÉSULTAT ATTENDU (LIGNE DE HORIZON-VM)

```
horizon-vm | ACTIVE | horizon-network=192.168.10.210, 172.24.4.92 | cirros... | m1.tiny
```

La **topologie réseau** de Horizon confirme l'architecture. Le réseau **horizon-network** (**192.168.10.0/24** , en rouge) est relié au réseau externe **public** (**172.24.4.0/24** , en bleu) par le routeur **horizon-router** , dont l'interface côté réseau privé est **192.168.10.1** . L'instance **horizon-vm** y porte l'adresse fixe **192.168.10.210** . Les réseaux **shared** et **private** et le routeur **router1** sont créés par défaut par DevStack.

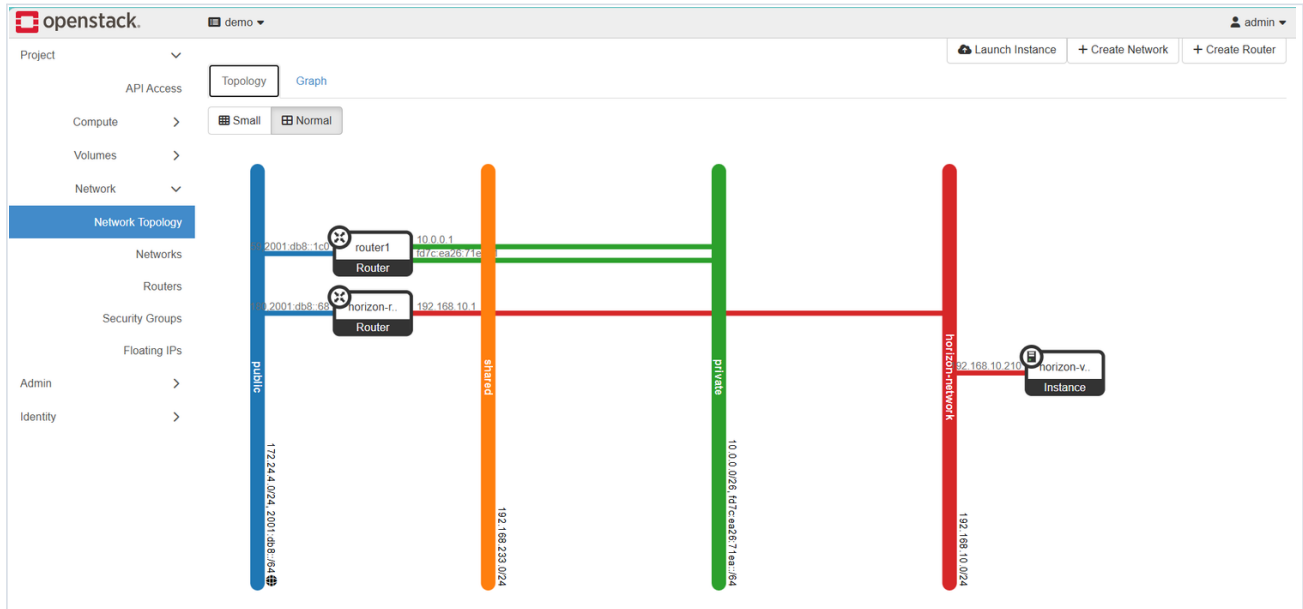


Figure 8 — Topologie réseau : horizon-network relié à public via horizon-router

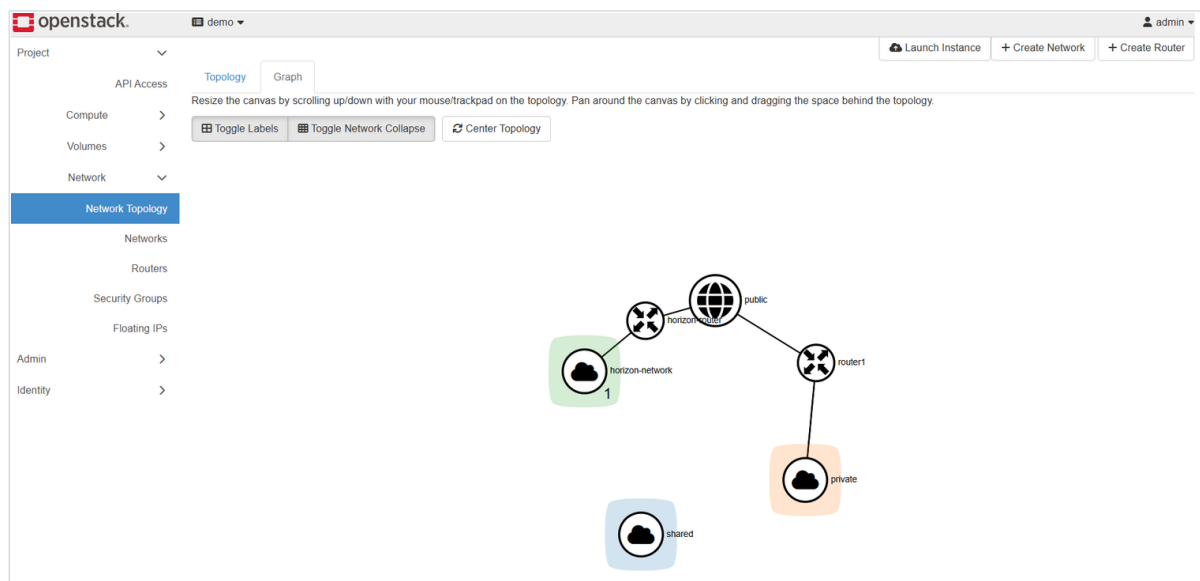


Figure 9 — Vue graphique de la topologie (horizon-router et router1 reliés à public)

Accès à la VM

La VM n'étant joignable que depuis son réseau privé, une IP flottante du pool **public** lui est associée (**172.24.4.92** dans la liste de la figure 10). Ce réseau n'étant accessible que depuis l'hôte EC2 (pont **br-ex**), la clé privée est copiée sur l'EC2 avec **scp** , puis la connexion se fait depuis l'hôte :

```
# Poste Windows
scp -i "C:\chemin\cle-aws.pem" "C:\chemin\cle-openstack.pem" ubuntu@<IP_EC2>:~/

# Sur l'EC2
chmod 400 ~/cle-openstack.pem
ssh -i ~/cle-openstack.pem cirros@<Floating_IP_de_la_VM>
```

COMMANDES (DEPUIS L'EC2)

```
ssh -i ~/cle-openstack.pem cirros@172.24.4.92
ip a
```

Résultat attendu : acceptation de l'empreinte à la première connexion, puis l'invite de la VM cirros ; `ip a` n'affiche que l'IP fixe `192.168.10.210`, l'IP flottante étant traduite par le routeur.

D Création d'une VM avec la CLI OpenStack

La même architecture est reconstruite en ligne de commande, avec le préfixe `cli-` et le sous-réseau `192.168.20.0/24`.

CRÉATION DES RESSOURCES

```
source ~/devstack/openrc admin admin

openstack network create cli-network
openstack subnet create cli-subnet --network cli-network --subnet-range 192.168.20.0/24

openstack router create cli-router
openstack router set cli-router --external-gateway public
openstack router add subnet cli-router cli-subnet

openstack security group create cli-sg
openstack security group rule create --protocol tcp --dst-port 22 cli-sg

openstack keypair create cli-keypair > cli-keypair.pem
chmod 400 cli-keypair.pem

openstack server create cli-vm --image "cirros-0.6.3-x86_64-disk" \
  --flavor m1.tiny --network cli-network \
  --security-group cli-sg --key-name cli-keypair
```

D.1 Allocation et association de l'IP flottante

L'adresse `172.24.4.93` est allouée dans le pool `public` : dans `openstack floating ip list`, elle n'a alors ni IP fixe ni port (`None`). L'adresse `172.24.4.92` est déjà associée à l'IP fixe `192.168.10.210`, celle de `horizon-vm`. La commande `openstack server add floating ip` lie ensuite `172.24.4.93` à `cli-vm`.

```
openstack floating ip create public
openstack floating ip list
openstack server add floating ip cli-vm 172.24.4.93
```

D.2 Connexion SSH à la VM

Depuis l'hôte EC2, la connexion à `172.24.4.93` avec la clé `cli-keypair.pem` aboutit : l'empreinte ED25519 de la VM est acceptée à la première connexion, puis l'invite de la VM cirros s'affiche. Les cinq éléments nécessaires à l'accès SSH (réseau privé, routeur, IP flottante, groupe de sécurité, paire de clés) sont donc en place.

```

ubuntu@ip-172-31-43-124:~/devstack$ openstack floating ip list
+-----+-----+-----+-----+-----+-----+
| ID | Floating IP Address | Fixed IP Address | Port | Floating Network | Project |
+-----+-----+-----+-----+-----+-----+
| 6fae865e-eb16-41ce-9fa7-f243b29dab27 | 172.24.4.93 | None | None | 148a0ca0-969f-439f-9e12-0023af77afc8 | 71275e5776de4b0aad3a5160b2c82694 |
| f7ed9a4e-0423-4980-9ab4-871c964d62d1 | 172.24.4.92 | 192.168.10.210 | 453c2119-3add-4766-ad8a-f44e78fad590 | 148a0ca0-969f-439f-9e12-0023af77afc8 | 2aad620377634ebdbed41ade8997e288 |
+-----+-----+-----+-----+-----+-----+

ubuntu@ip-172-31-43-124:~/devstack$ openstack server add floating ip cli-vm 172.24.4.93
ubuntu@ip-172-31-43-124:~/devstack$ ssh -i cli-keypair.pem cirros@172.24.4.93
The authenticity of host '172.24.4.93 (172.24.4.93)' can't be established.
ED25519 key fingerprint is: SHA256:oFFPIGJJRdhG8iI3oTJ+/YNI7uu9nFX1Zt8SdGU4Fmc
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.24.4.93' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
$

```

Figure 10 — Liste des IP flottantes, association à cli-vm et connexion SSH en tant que cirros

D.3 État des instances

`openstack server list` montre `cli-vm` avec ses deux adresses : l'IP fixe `192.168.20.38` sur `cli-network` et l'IP flottante `172.24.4.93`. Le statut est `SHUTOFF` au moment de la capture : la VM avait été arrêtée après les tests. Elle se relance avec `openstack server start cli-vm`.

```

ubuntu@ip-172-31-43-124:~/devstack$ openstack server list
+-----+-----+-----+-----+-----+-----+
| ID | Name | Status | Networks | Image | Flavor |
+-----+-----+-----+-----+-----+-----+
| fa5427e7-d5c2-4091-9356-b9c2a9e15cff | cli-vm | SHUTOFF | cli-network=172.24.4.93, 192.168.20.38 | cirros-0.6.3-x86_64-disk | m1.tiny |
+-----+-----+-----+-----+-----+-----+

```

Figure 11 — `openstack server list` : cli-vm (cirros, m1.tiny) avec IP fixe et flottante

D.4 Interfaces réseau sur l'hôte

Sur l'hôte, `ip a` révèle des interfaces `tap...` rattachées à `ovs-system`. Ce sont les ports créés par le réseau OpenStack (Open vSwitch / OVN) dans des espaces de noms `ovnmeta-...`, un par réseau hébergeant des VM. Leur MTU de 1442 octets, inférieur aux 1500 habituels, s'explique par l'encapsulation Geneve du réseau de recouvrement.

```

9: tapcdc79ed9-f0@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1442 qdisc noqueue master ovs-system state UP group default qlen 1000
    link/ether c2:08:63:68:08:12 brd ff:ff:ff:ff:ff:ff link-netns ovnmeta-cdc79ed9-fc27-4426-a359-9f787e73c8f5
    inet6 fe80::c008:63ff:fe68:812/64 scope link proto kernel_ll
    valid_lft forever preferred_lft forever
10: tapfd384dcf-90@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1442 qdisc noqueue master ovs-system state UP group default qlen 1000
    link/ether 26:7c:bf:9c:f4:d4 brd ff:ff:ff:ff:ff:ff link-netns ovnmeta-fd384dcf-9424-4a31-917d-65093c282791
    inet6 fe80::247c:bfff:fe9c:f4d4/64 scope link proto kernel_ll
    valid_lft forever preferred_lft forever
ubuntu@ip-172-31-43-124:~/devstack$

```

Figure 12 — Interfaces tap rattachées à ovs-system sur l'hôte (ports de métadonnées OVN)

E Analyse et conclusion

Points d'architecture à retenir

- **Deux niveaux de virtualisation** : l'hyperviseur d'AWS héberge l'hôte EC2, et KVM y héberge les VM d'OpenStack. Sans virtualisation imbriquée, ce second niveau n'est pas accéléré.
- **Réseau externe isolé** : sous DevStack, le réseau `public` n'est relié qu'au pont `br-ex` de l'hôte. Les IP flottantes ne sont donc joignables que depuis l'EC2.
- **Cinq éléments pour un accès SSH** : réseau privé et sous-réseau, routeur relié à `public`, IP flottante, groupe de sécurité autorisant le port 22, paire de clés.
- **Horizon et CLI sont équivalents** : ils appellent les mêmes API. Horizon aide à visualiser (topologie), la CLI est reproductible et scriptable.

Bonnes pratiques et coûts

- Restreindre la source SSH de `DevStack SG` à une adresse IP connue plutôt que `0.0.0.0/0`, et remplacer le mot de passe `secret` en dehors d'un TP.
- Ne jamais versionner les clés privées `.pem` (exclues par le `.gitignore`).
- Arrêter l'instance après le TP et libérer l'IP élastique : l'IP élastique et le volume EBS restent facturés même instance arrêtée.

Conclusion

Le TP a permis de déployer de bout en bout une plateforme IaaS : de l'instance EC2 et de la virtualisation imbriquée jusqu'à la création de machines virtuelles accessibles en SSH, par l'interface Horizon comme par la CLI. Il met en évidence le rôle de chaque service OpenStack (Keystone, Nova, Neutron, Glance) et la logique réseau qui relie une VM privée au monde extérieur.